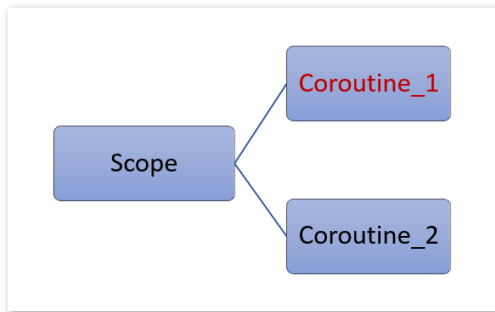


Урок 14. Корутины. Обработка исключений. Вложенные корутины

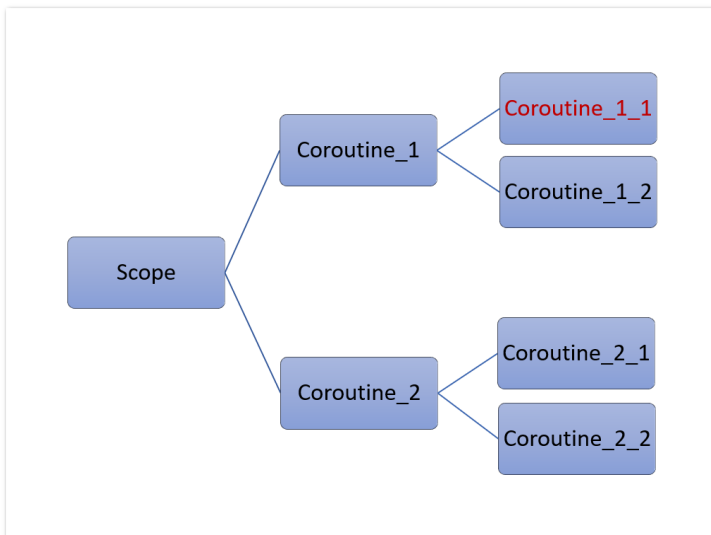
05 июня 2020

В этом уроке продолжаем обсуждать обработку исключений на примере вложенных корутин.

В прошлом уроке мы рассмотрели обработку ошибок в простых случаях, когда ошибка происходила в корутине, которая вызывалась из scope.



Теперь рассмотрим случай посложнее, когда ошибка происходит в корутине, вызванной из другой корутины.



Идти будем примерно по тому же плану, что и в прошлом уроке. В качестве диспетчера используйте **Dispatchers.IO**, чтобы было доступно больше потоков.

try-catch

Тут ничего не поменялось.

Если код ошибки поместить в try-catch, то он будет пойман и ничего не случится.

Пример:

```
1 | scope.launch { // Coroutine_1
2 |     launch { // Coroutine_1_1
3 |         try { Integer.parseInt("a") } catch (e: Exception) {}
4 |     }
5 |     launch {} // Coroutine_1_2
6 | }
7 |
8 | scope.launch { // Coroutine_2
9 |     launch {} // Coroutine_2_1
10 |    launch {} // Coroutine_2_2
11 | }
```

Ошибка в корутине 1_1 будет сразу поймана и ни на что не повлияет.

А если в try-catch поместить вызов билдера launch, то это никак не поможет поймать ошибку:

```

1  scope.launch { // Coroutine_1
2      try {
3          launch { Integer.parseInt("a") } // Coroutine_1_1
4      } catch (e: Exception) {}
5      launch {} // Coroutine_1_2
6  }
7
8  scope.launch { // Coroutine_2
9      launch {} // Coroutine_2_1
10     launch {} // Coroutine_2_2
11 }

```

Причины те же, что и раньше. Билдер обрабатывает без ошибок, он создает и корутину и отправляет ее на запуск в диспетчер. Поэтому try-catch ничего не ловит. А вот когда диспетчер отправляет корутину выполняться, то там уже во время выполнения происходит ошибка. Она ловится внутренними механизмами Continuation и с помощью Job передается родителю. И вот здесь начинаются отличия от прошлого урока.

Передача ошибки родителю

В прошлом уроке мы рассматривали, что при возникновении ошибки корутину сообщает об этом родителю. Он отменяет всех своих детей. А корутину пытается сама обработать ошибку.

Это был немного упрощенный алгоритм действий корутины. В нем не хватает одного важного этапа взаимодействия между родителем и корутиной. Когда корутину сообщает родителю об ошибке, она передает ему эту ошибку и спрашивает, сможет ли родитель ошибку обработать. Ответ родителя зависит от того, кем этот родитель является - **скоупом** или **корутиной**.

Если родитель - **скоуп**, то он отвечает, что не сможет. Поэтому корутину пытается это сделать сама. Об этом мы подробно говорили в прошлом уроке. Она отправляет ошибку в обработчик CoroutineExceptionHandler. А если такого обработчика ей не предоставили, то ошибка уходит в глобальный обработчик, и приложение падает.

А если родитель - **корутина**, то она отвечает, что сможет. И дочерняя корутину вместо всевозможных обработчиков передает ошибку родительской корутине.

Родительская корутину, хоть и сказала, что обработать сможет, но сначала сама спрашивает у своего родителя, сможет ли он обработать. И если может, то передает ему ошибку.

Таким образом эта ошибка поднимается вверх по иерархии вложенных корутин, пока не достигнет самой верхней корутины. Ее родитель - это скоуп. Когда самая верхняя корутину спросит его, он скажет, что ошибку обработать не сможет. И в итоге самая верхняя корутину отправляет ошибку в обработчики, как мы рассматривали в прошлом уроке.

При этом каждый родитель, через который прошла ошибка, будет отменяться сам и отменять все свои дочерние корутины. Тут поведение не отличается от скоупа.

CoroutineExceptionHandler

Из объяснений выше следует, что обработчик CoroutineExceptionHandler сработает только в самой верхней корутине. Потому что только она получит от своего родителя отрицательный ответ и попытается обработать ошибку сама. Остальные корутины просто передают ошибку родительской корутине и сами ничего с ней делают.

Давайте убедимся в этом на примере.

```

1  val handler = CoroutineExceptionHandler { context, exception ->
2      log("$exception was handled in Coroutine_${context[CoroutineName]?.name}")
3  }
4
5  val scope = CoroutineScope(Job() + Dispatchers.Default + handler)
6
7  scope.launch(CoroutineName("1")) {
8
9      launch(CoroutineName("1_1")) {
10         TimeUnit.MILLISECONDS.sleep(500)
11         Integer.parseInt("a")
12     }
13
14     launch(CoroutineName("1_2")) {
15         TimeUnit.MILLISECONDS.sleep(1000)
16     }
17 }
18
19 scope.launch(CoroutineName("2")) {
20
21     launch(CoroutineName("2_1")) {
22         TimeUnit.MILLISECONDS.sleep(1000)
23     }
24
25     launch(CoroutineName("2_2")) {
26         TimeUnit.MILLISECONDS.sleep(1000)
27     }
28 }

```

CoroutineName - это элемент контекста. Помещаем в него имя корутины и передаем в launch билдеры. Теперь в каждой корутине мы сможем из контекста достать этот элемент и узнать, какая это корутина.

В обработчике CoroutineExceptionHandler мы на вход получаем ошибку и контекст корутины, которая передала ошибку в этот обработчик. Мы из контекста достаём CoroutineName и с его помощью узнаем, какая корутина обработала ошибку.

Обработчик мы помещаем в scope, поэтому он будет передан во все корутины.

В корутине 1_1 выбросим исключение через 500 мсек. Остальные корутины просто ничего не делают 1000 мсек.

Запускаем код, смотрим лог:

java.lang.NumberFormatException: For input string: "a" was handled in Coroutine_1

Ошибка была обработана в корутине 1. Все в соответствии с алгоритмом. Корутина 1_1, в которой произошла ошибка, сама не стала его обрабатывать, а передала родителю - корутине 1. А тот не смог передать ошибку дальше своему родителю, потому что это scope, и отправил ее в свой обработчик.

В качестве упражнения и закрепления темы попробуйте создать еще несколько дочерних корутин, увеличив уровень вложенности. И там вызовите ошибку. Проверьте, что она все равно придет в самую верхнюю корутину и обработается там.

Отмена

Давайте проверим, что ошибка по пути вверх поотменяет все корутины.

Сначала напишем небольшое расширение для CoroutineScope. Мы сможем запускать его прямо в корутине. Оно будет периодически выводить в лог имя и isActive статус корутины.

```
1 fun CoroutineScope.repeatIsActive() {
2     repeat(5) {
3         TimeUnit.MILLISECONDS.sleep(300)
4         log("Coroutine_${coroutineContext[CoroutineName]?.name} isActive $isActive")
5     }
6 }
```

По правилам мы, конечно, должны сразу завершать работу корутины, если isActive возвращает false. Мы обсуждали это в уроке про отмену корутин. Но здесь я этого не делаю, чтобы не усложнять код.

Перепишем пример. Корутина 1_1 вызовет ошибку через 1000 мсек. А остальные 5 корутин в это время пишут в лог свое состояние методом repeatIsActive.

```
1 scope.launch(CoroutineName("1")) {
2
3     launch(CoroutineName("1_1")) {
4         TimeUnit.MILLISECONDS.sleep(1000)
5         log("exception")
6         Integer.parseInt("a")
7     }
8
9     launch(CoroutineName("1_2")) { repeatIsActive() }
10
11     repeatIsActive()
12 }
13
14 scope.launch(CoroutineName("2")) {
15
16     launch(CoroutineName("2_1")) { repeatIsActive() }
17
18     launch(CoroutineName("2_2")) { repeatIsActive() }
19
20     repeatIsActive()
21 }
```

Запускаем, смотрим лог:

```
19:08:47.961 Coroutine_2 isActive true
19:08:47.961 Coroutine_1 isActive true
19:08:47.964 Coroutine_1_2 isActive true
19:08:47.964 Coroutine_2_1 isActive true
19:08:47.968 Coroutine_2_2 isActive true

19:08:48.264 Coroutine_2 isActive true
19:08:48.264 Coroutine_1 isActive true
19:08:48.264 Coroutine_1_2 isActive true
19:08:48.265 Coroutine_2_1 isActive true
19:08:48.270 Coroutine_2_2 isActive true

19:08:48.566 Coroutine_1 isActive true
19:08:48.566 Coroutine_2_1 isActive true
19:08:48.566 Coroutine_1_2 isActive true
19:08:48.566 Coroutine_2 isActive true
19:08:48.572 Coroutine_2_2 isActive true

19:08:48.663 exception
```

```

19:08:48.869 Coroutine_1 isActive false
19:08:48.869 Coroutine_2 isActive false
19:08:48.869 Coroutine_2_1 isActive false
19:08:48.869 Coroutine_1_2 isActive false
19:08:48.874 Coroutine_2_2 isActive false

19:08:49.172 Coroutine_1 isActive false
19:08:49.172 Coroutine_2_1 isActive false
19:08:49.175 Coroutine_1_2 isActive false
19:08:49.175 java.lang.NumberFormatException: For input string: "a" handled in Coroutine_1
19:08:49.184 Coroutine_2_2 isActive false
19:08:49.172 Coroutine_2 isActive false

```

Я его разделил пустыми строками на части, чтобы легче было читать. Поначалу все корутины активны. Но после того, как в корутине 1_1 была вызвана ошибка, все остальные корутины этого scope были отменены.

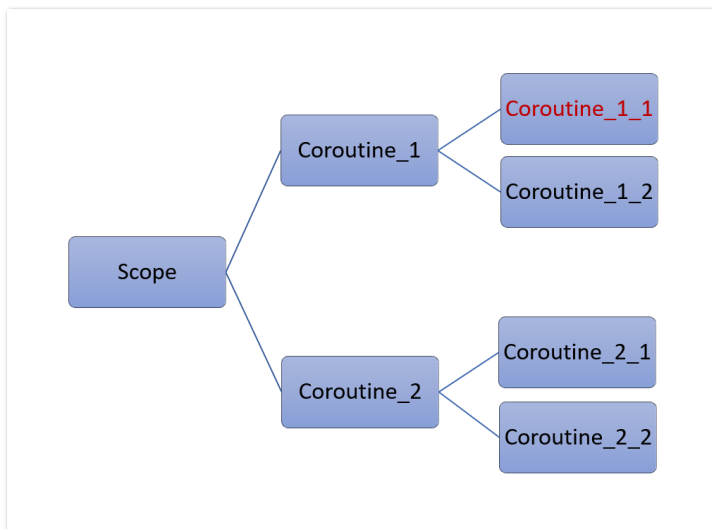
Т.е. ошибка поднимается вверх до scope, отменяя все, что можно. И далее распространяется на остальные корутины этого scope, каскадно отменяя в них все дочерние корутины.

SupervisorJob

В прошлом уроке мы использовали SupervisorJob в качестве Job в scope. Это приводило к тому, что scope не отменял все свои корутины, когда получал ошибку от одной из них.

В случае с вложенными корутинами SupervisorJob в scope нам не особо поможет. Ошибка пройдет несколько родительских корутин, прежде чем доберется до scope. И все эти родительские корутины будут отменяться сами и отменяют свои дочерние корутины.

Взглянем снова на структуру корутин:



В корутине 1_1 происходит ошибка. Ошибка идет в корутину 1. Эта корутина отменяет себя и корутину 1_2. Далее ошибка идет в scope. Если там SupervisorJob, то scope не станет отменять корутину 2, а значит не будут отменены и корутины 2_1 и 2_2.

Т.е. SupervisorJob в scope помогает только частично. Он останавливает распространение ошибки на другие корутины этого scope. В нашем случае - это корутина 2 и все ее дочерние корутины.

Можем ли мы использовать SupervisorJob в качестве Job в каждой корутине, чтобы ошибка вообще не распространялась?

Т.е. можно ли сделать как-то так?

```

1 | scope.launch { // parent coroutine
2 |
3 |     launch(SupervisorJob()) { // child coroutine
4 |
5 |     }
6 | }

```

Когда мы передаем какой-либо Job в билдер launch, то этот Job будет использован, как родитель создаваемой корутины. Т.е. мы подменяем родительский Job в контексте дочерней корутины. Это приведет к тому, что Job дочерней корутины не установит связь с реальным Job-ом родительской корутины.

Т.е. передав SupervisorJob в билдер, создающий дочернюю корутину мы вместо связи:

parentJob > *childJob*

получим:

supervisorJob > *childJob*

А это уже нарушение принципа structured concurrency и связи между корутинами.

Есть костыль, который можно использовать, чтобы сохранить связь:

```
1 | scope.launch { // parent coroutine
2 |
3 |     launch(SupervisorJob(coroutineContext[Job])) { // child coroutine
4 |
5 |     }
6 | }
```

Мы в SupervisorJob передаем Job родительской корутины. Это приведет к тому, что этот родительский Job станет родителем SupervisorJob. А, т.к. сам SupervisorJob станет родителем для Job-а дочерней корутины, то получится такая связь:

parentJob > supervisorJob > childJob

Получилось три джоба на две корутины. Но зато связь между корутинами сохраняется - при отмене родителя отменятся и его дочерние корутины.

Если теперь в дочерней корутине произойдет ошибка, то childJob передаст ее в supervisorJob, а тот ничего не будет отменять и дальше (в parentJob) эту ошибку передавать не будет. Т.е. ошибка будет направлена в обработчик дочерней корутиной.

Способ хоть и рабочий, но достаточно странно выглядящий. Есть другое решение - оператор supervisorScope. Его мы подробно обсудим в отдельном уроке.